

INVENTORY & STOCK MANAGEMENT SYSTEM

Submitted By: Kanishka

Registration No.: 25BAI10070

Branch: B.Tech - Computer Science &
Engineering (AIML)

Course: Object-Oriented
Programming (Java)

Evaluation Platform: VITyarthi

Date of Submission: September
2026

1. INTRODUCTION

In retail and supply chains, manual inventory handling often leads to human recording mistakes, overstocking, and stockouts. This project provides an automated, menu-driven Java console application to track catalog products, inventory movements, safety levels, and supplier data.

2. PROBLEM STATEMENT

Small-to-medium businesses face revenue loss and order delays due to untracked stock dispatching and the absence of safety threshold alerts. This project addresses this by implementing an in-memory tracking framework with transactional logging and stock validations.

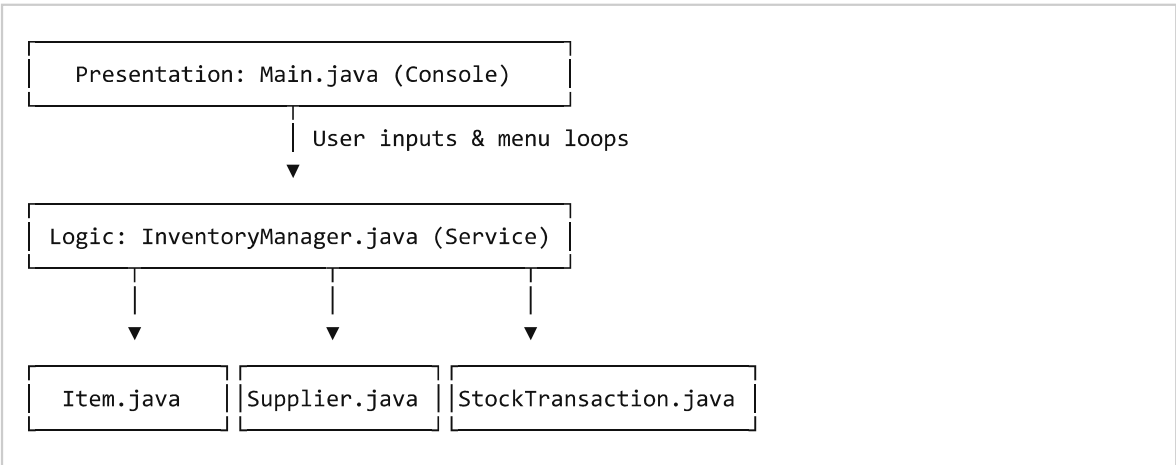
3. FUNCTIONAL REQUIREMENTS

- **Item Management:** Register products with SKU identifiers, unit prices, threshold levels, and link them to specific suppliers.
- **Stock Movement Tracking:** Record inward restocks and outward dispatches with validations to prevent negative stock.
- **Alerting & Valuation:** Auto-filter items below safety thresholds and calculate the total financial holding value of inventory.

4. NON-FUNCTIONAL REQUIREMENTS

- **Usability:** Fast, numbered terminal menu interface requiring zero GUI setups.
- **Reliability:** Input and balance validations block invalid dispatches when stock is insufficient.
- **Maintainability:** Modular architecture separated across dedicated model, transaction, and manager classes.
- **Resource Efficiency:** Lightweight in-memory storage using standard Java Collections (ArrayList).

5. SYSTEM ARCHITECTURE

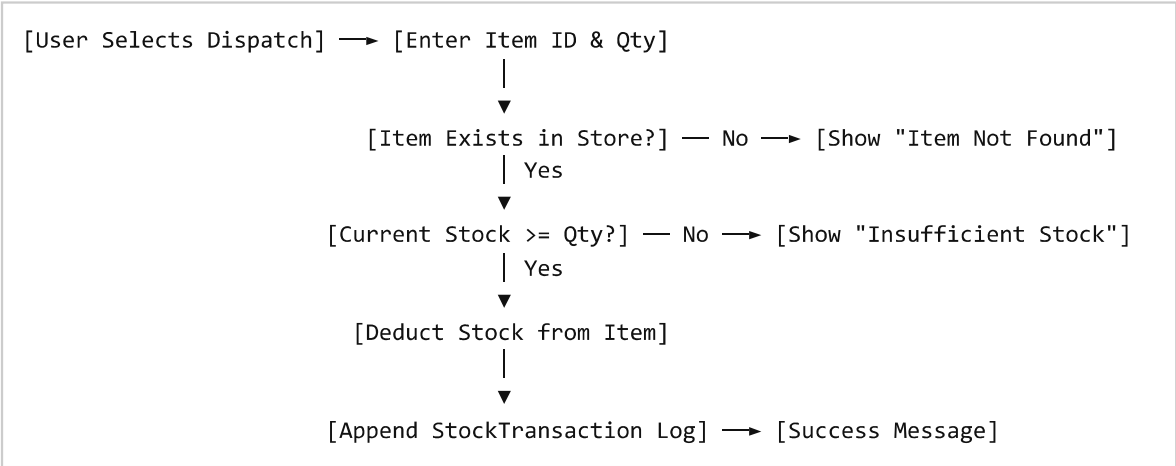


6. DESIGN DIAGRAMS

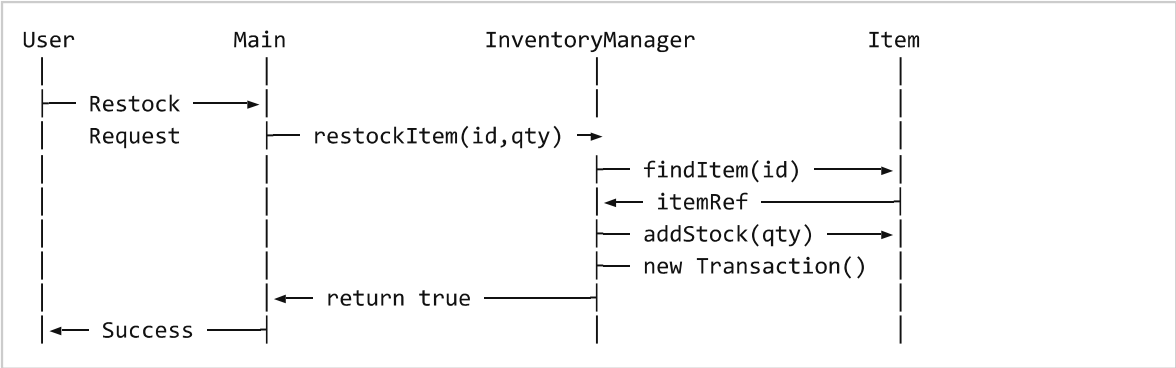
Use Case Diagram



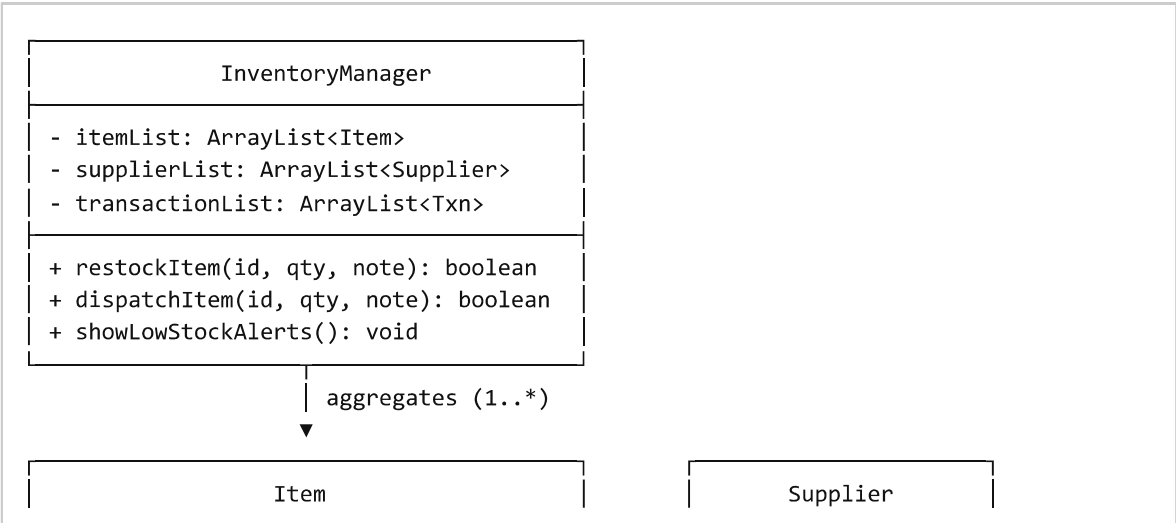
Workflow Diagram

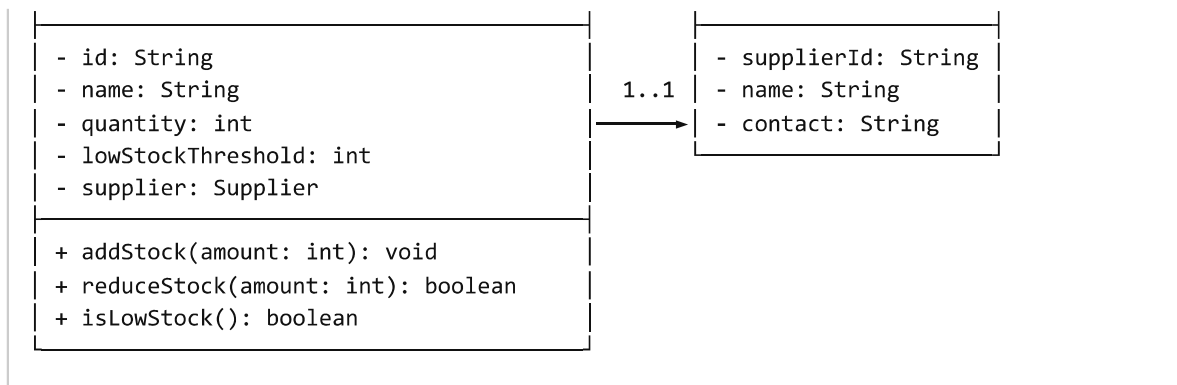


Sequence Diagram

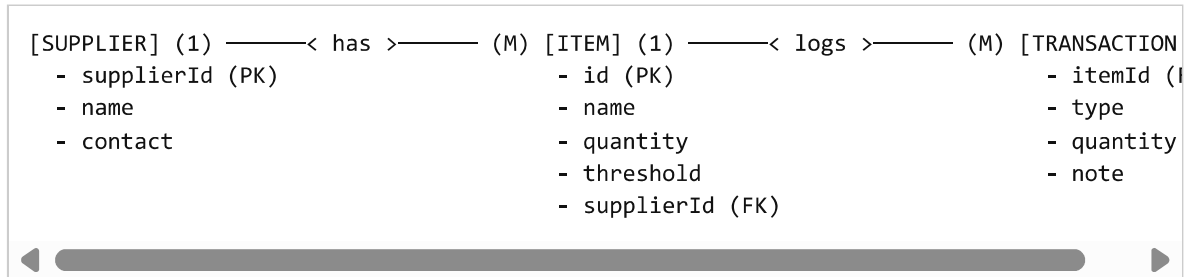


Class / Component Diagram





ER Diagram (In-Memory Schema)



7. DESIGN DECISIONS & RATIONALE

- **Object Composition:** Binding Supplier directly to Item ensures relational integrity without needing an external database.
- **Defensive Reduction:** reduceStock() returns a boolean, ensuring the application blocks illegal balance deductions.
- **Console-Only I/O:** Built using Scanner and standard output to guarantee complete command-line compatibility.

8. IMPLEMENTATION DETAILS

The code is divided into 5 focused files:

- Supplier.java: Vendor contact and identity structure.
- Item.java: Core inventory logic, stock addition, and threshold checks.
- StockTransaction.java: Audit record structure tracking stock changes.
- InventoryManager.java: Business logic, searches, and reporting math.
- Main.java: Menu loop and user input parsing.

9. SCREENSHOTS / RESULTS

1. Main Menu & Initial Inventory Catalog

```
Run Main x
"\"C:\Program Files\Java\jdk-26.0.1\bin\java.exe\" \"-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=58767\" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=...

=====
INVENTORY & STOCK MANAGER
=====
1. View All Items
2. Add New Item
3. Restock Item (Inward)
4. Dispatch / Sell Item (Outward)
5. View Low Stock Alerts
6. Reports (Valuation & History)
7. Exit
Enter choice (1-7): 1

--- Current Inventory ---
ID: ITM1 | Name: Mechanical Keyboard | Stock: 12 | Min Threshold: 5 | Price: $50.0 | Supplier: Apex Logistics
ID: ITM2 | Name: USB Cable | Stock: 3 | Min Threshold: 5 | Price: $9.5 | Supplier: Tech Supplies
```

2. Inward Stock Restock Operation

```
=====
INVENTORY & STOCK MANAGER
=====
1. View All Items
2. Add New Item
3. Restock Item (Inward)
4. Dispatch / Sell Item (Outward)
5. View Low Stock Alerts
6. Reports (Valuation & History)
7. Exit
Enter choice (1-7): 3
Enter Item ID to restock: ITM1
Enter Quantity to add: 10
Enter Note / Batch reference: Batch delivery from Apex
Restock successful.
```

3. Outward Dispatch Validation (Insufficient Stock Error)

```
=====
INVENTORY & STOCK MANAGER
=====
1. View All Items
2. Add New Item
3. Restock Item (Inward)
4. Dispatch / Sell Item (Outward)
5. View Low Stock Alerts
6. Reports (Valuation & History)
7. Exit
Enter choice (1-7): 4
Enter Item ID to dispatch: ITM2
Enter Quantity to dispatch: 50
Enter Reason / Customer name: Bulk order
Failed: Insufficient stock or item not found.
```

4. Automated Real-Time Low Stock Alert

```
=====
INVENTORY & STOCK MANAGER
=====
1. View All Items
2. Add New Item
3. Restock Item (Inward)
4. Dispatch / Sell Item (Outward)
5. View Low Stock Alerts
6. Reports (Valuation & History)
7. Exit
Enter choice (1-7): 5

--- Low Stock Alerts ---
ALERT: USB Cable is low! Current: 3 (Threshold: 5)
```

5. Cumulative Asset Valuation & Movement Audit History

```
=====
INVENTORY & STOCK MANAGER
=====
1. View All Items
2. Add New Item
3. Restock Item (Inward)
4. Dispatch / Sell Item (Outward)
5. View Low Stock Alerts
6. Reports (Valuation & History)
7. Exit
Enter choice (1-7): 6
Total Inventory Asset Value: $1125.5

--- Stock Movement History ---
Type: RESTOCK | Item ID: ITM1 | Quantity: 12 | Note: Initial Inventory
Type: RESTOCK | Item ID: ITM2 | Quantity: 3 | Note: Initial Inventory
Type: RESTOCK | Item ID: ITM1 | Quantity: 10 | Note: Batch delivery from Apex
```

10. TESTING APPROACH

- **Boundary Tests:** Attempted to dispatch more stock than available to confirm the system prevented negative inventory.
- **Threshold Verification:** Confirmed that reducing stock to or below the safety value triggered alerts.
- **Input Handling:** Handled invalid menu selections gracefully without runtime crashes.

11. CHALLENGES FACED

- **Scanner Buffer Skipping:** Fixed line-skipping issues by clearing newline characters after numerical inputs.
- **Log Consistency:** Configured dispatching logic so movement history logs are generated only after balance validations succeed.

12. LEARNINGS & KEY TAKEAWAYS

- Applied core Object-Oriented principles (Encapsulation, Aggregation, Separation of Concerns) to a real-world problem domain.
- Gained practical experience with Java Collections (ArrayList) and defensive logic patterns.

13. FUTURE ENHANCEMENTS

- Adding file persistence (JSON or CSV) or an SQL database to retain state across restarts.
- Implementing role-based access control for administrative vs. store-floor users.

14. REFERENCES

1. Oracle Java Documentation: Collections Framework & Scanner API.
2. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill.
3. Course Syllabus & Project Evaluation Rubric, VITYarthi Portal.